

8장 프로젝트 실무 적용

이번 장은 금융 서비스와 같은 실무형 화면 구현부터 팀 단위 미니 프로젝트 완성까지, React 기반 화면 설계, API 연동, 조건부 렌더링, 예외 처리, 데이터 바인딩, 발표 및 피드백 기반 개선까지 전체 프로젝트 수행 과정을 경험하며 실무 수준의 프론트엔드 개발 능력과 협업 역량을 동시에 향상시키는 장이다.

8.1 금융 서비스 목록·상세 화면 구현

금융 서비스 화면 구현 절에서는 목록과 상세 화면 UI를 설계하고 외부 API와 연동하여 실시간 데이터를 바인딩하며, 조건부 렌더링과 예외 처리를 적용해 다양한 사용자 시나리오를 처리하고, 실제 유사 금융 화면 결과물을 제작하여 실무형 화면 설계 및 구현 능력을 익히는 과정을 다룬다.

(1) 목록/상세 UI 구조 설계

목록과 상세 화면의 UI 구조를 컴포넌트 단위로 설계하고, 카드·테이블·리스트 구조를 적절히 활용하여 정보 표시와 화면 전환의 흐름을 직관적으로 구성하며, 재사용 가능한 컴포넌트와 상태 관리를 고려한 설계 전략을 학습한다.

① 목록 컴포넌트 설계

목록 화면은 각 금융 서비스 항목을 카드나 리스트 아이템으로 표시하며, 반복되는 UI 구조를 컴포넌트화하여 재사용성을 높이고, props로 데이터 전달과 이벤트 핸들링을 효율적으로 처리한다.

```
// src/components/ServiceList.js
import React from 'react';
import ServiceItem from './ServiceItem';

function ServiceList({ services, onSelect }) {
  if (!services || services.length === 0) return <p>서비스가 없습니다.</p>;

  return (
    <div className="service-list">
      {services.map(service => (
```

```

        <ServiceItem key={service.id} service={service} onSelect={onSelect}
      />
    )))
  </div>
);
}

export default ServiceList;

```

② 상세 화면 컴포넌트 설계

선택한 항목의 상세 정보를 보여주는 컴포넌트로, 상위에서 선택된 서비스 데이터를 props로 받아 표시하고, 버튼이나 이벤트를 통해 상태 변경과 화면 전환을 관리한다.

```

// src/components/ServiceDetail.js
import React from 'react';

function ServiceDetail({ service, onBack }) {
  if (!service) return <p>선택된 서비스가 없습니다.</p>;

  return (
    <div className="service-detail">
      <h2>{service.name}</h2>
      <p>계좌 번호: {service.account}</p>
      <p>잔액: {service.balance}원</p>
      <button onClick={onBack}>목록으로 돌아가기</button>
    </div>
  );
}

export default ServiceDetail;

```

(2) 연동 데이터 바인딩 - API

외부 금융 API 또는 모의 데이터를 이용해 목록과 상세 화면에 실시간 데이터를 바인딩하고, fetch/axios를 활용한 비동기 호출과 상태 업데이트, 로딩·에러 처리 방식을 적용하여 화면과 데이터의 동기화를 실무 수준으로 구현한다.

① 데이터 fetch 및 상태 관리

```
// src/App.js
import React, { useState, useEffect } from 'react';
import ServiceList from './components/ServiceList';
import ServiceDetail from './components/ServiceDetail';
import axios from 'axios';

function App() {
  const [services, setServices] = useState([]);
  const [selectedService, setSelectedService] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    axios.get('https://api.example.com/services')
      .then(res => setServices(res.data))
      .catch(err => setError('데이터 로딩 실패'))
      .finally(() => setLoading(false));
  }, []);

  return (
    <div>
      {loading && <p>로딩 중...</p>}
      {error && <p>{error}</p>}
      {!selectedService
        ? <ServiceList services={services} onSelect={setSelectedService} />
        : <ServiceDetail service={selectedService} onBack={() => setSelectedService(null)} />}
    </div>
  );
}
```

```
export default App;
```

② axios와 fetch 비교

- **fetch**: 브라우저 내장, 가볍고 Promise 기반, 별도 설치 필요 없음
- **axios**: JSON 자동 변환, 요청/응답 인터셉터, 타임아웃 설정 가능, 실무에서 선호

```
// fetch 예시
fetch('https://api.example.com/services')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

(3) 조건부 화면 렌더링 및 예외 처리

사용자 입력, 데이터 유무, 네트워크 상태 등 다양한 조건에 따라 화면을 다르게 렌더링하고, 오류 메시지, 빈 데이터 처리, 로딩 스피너 등 예외 처리 UI를 구현하여 안정적이고 사용자 친화적인 화면 흐름을 구성하는 기법을 학습한다.

① 조건부 렌더링 적용

```
{loading && <p>로딩 중...</p>}
{error && <p>{error}</p>}
{!selectedService ? <ServiceList ... /> : <ServiceDetail ... />}
```

② 빈 데이터 처리

```
if (!services || services.length === 0) return <p>등록된 서비스가 없습니다.</p>
>;
```

③ 네트워크 오류 처리

```
.catch(err ⇒ setError('API 호출 실패, 다시 시도해주세요'))
```

(4) 실무 유사 화면 결과물 제작

설계된 UI와 연동 데이터를 기반으로 실무 금융 서비스와 유사한 화면을 완성하고, 목록 조회·상세 보기·상태 변경 등 핵심 기능을 통합하여 완성도 높은 프로토타입을 제작하며, 팀/개인 프로젝트 결과물로 실무 경험을 쌓는다.

① 화면 전환

```
<ServiceList services={services} onSelect={setSelectedService} />
<ServiceDetail service={selectedService} onBack={() ⇒ setSelectedService(null)} />
```

② 스타일 적용

```
.service-list { display: flex; flex-wrap: wrap; gap: 16px; }
.service-item { border: 1px solid #ddd; padding: 12px; border-radius: 8px; cursor: pointer; }
.service-detail { padding: 20px; border: 1px solid #ccc; border-radius: 8px; }
```

③ 사용자 시나리오 통합

- 목록에서 서비스 선택 → 상세 화면 표시
- 상세 화면에서 뒤로가기 → 목록 화면 복귀
- API 로딩, 실패, 빈 데이터 처리 포함

8.2 미니 프로젝트 완성 및 발표

미니 프로젝트 완성 및 발표 절에서는 기능 기획서 작성과 구현 전략 수립부터 화면 설계, 기능 구현, 발표 자료 제작 및 시연, 그리고 피드백 기반 코드 개선과 문서화를 거쳐 실무 프로젝트 경험을 종합적으로 체험하고 발표 능력과 협업 능력을 동시에 강화하는 과정을 다룬다.

(1) 기능 기획서 작성 및 구현 전략 수립

프로젝트의 목표와 기능 요구사항을 정리한 기획서를 작성하고, 화면 흐름, API 연동, 상태 관리, 컴포넌트 구조 등 구현 전략을 수립하여 개발 전 단계에서 전체 아키텍처와 개발 계획을 체계적으로 설계한다.

① 요구사항 정의

각 기능의 입력/출력, 화면 구성, 예외 상황을 정의하여 개발 범위와 우선순위를 명확히 한다.

요구사항

- 서비스 목록 조회
- 상세 정보 보기
- 신규 항목 추가/삭제
- 데이터 동기화 및 상태 표시

② 화면 흐름 설계

사용자가 이동할 화면과 페이지 전환 흐름을 다이어그램 또는 간단한 UI 스케치를 통해 시각화한다.

화면 흐름:

목록 화면 → 상세 화면 → 편집 화면 → 목록 화면

③ 구현 전략 수립

API 구조, 상태 관리 위치, 컴포넌트 재사용 계획, 로딩/에러 처리 전략 등을 포함하여 개발 전반의 로드맵을 정의한다.

(2) 팀/개인별 화면 설계 및 기능 구현

팀 또는 개인별로 화면 구성과 UI 컴포넌트를 설계하고, 기획서에 기반하여 목록, 상세, 입력, 편집 등 기능을 React 컴포넌트 단위로 구현하며, API 연동, 상태 관리, 이벤트 처리 등 실제 동작하는 화면을 완성한다.

① 컴포넌트 설계

목록, 상세, 입력, 편집 등 화면 단위 컴포넌트를 설계하고, props와 state를 활용하여 데이터 흐름을 정의한다.

```
// src/components/ItemCard.js
function ItemCard({ item, onClick }) {
  return <div className="item-card" onClick={() => onClick(item)}>{item.name}</div>;
}
```

② 상태 관리 및 API 연동

useState, useEffect 또는 Context를 활용해 상태를 관리하고, fetch/axios로 데이터를 동기화하며 로딩과 에러를 처리한다.

```
useEffect(() => {
  axios.get('/api/items')
    .then(res => setItems(res.data))
    .catch(err => setError('데이터 로딩 실패'));
}, []);
```

③ 이벤트 처리 및 기능 구현

추가, 삭제, 수정 버튼 클릭 시 상태를 업데이트하고 UI를 갱신하며, 조건부 렌더링을 통해 사용자 경험을 향상시킨다.

```
const handleDelete = id ⇒ setItems(items.filter(i ⇒ i.id !== id));
```

(3) 발표자료 작성 및 시연 발표

구현된 프로젝트 화면과 기능을 바탕으로 발표 자료를 작성하고, 시연 발표를 통해 화면 흐름, 핵심 기능, 구현 전략 등을 설명하며, 청중의 이해와 피드백을 통해 프로젝트 완성도를 검증하고 발표 경험을 쌓는다.

① 발표 자료 구성

화면 캡처, 기능 설명, 개발 전략, 문제 해결 사례 등을 포함한 PPT 또는 PDF를 제작한다.

② 시연 발표

프로젝트를 직접 실행하며 기능 시연, 화면 이동, 상태 변경 등 핵심 기능을 보여주고, 실무 발표와 동일하게 질문에 대응한다.

(4) 피드백 기반 코드 수정 및 문서화

발표 및 팀 피드백을 바탕으로 UI·기능·코드 품질을 개선하고, README, 설계 문서, API 사용 설명 등 프로젝트 문서를 정리하여 유지보수성과 재사용성을 높이며, 최종 산출물을 완성하는 실무형 프로젝트 마무리 과정을 학습한다.

① 코드 개선

중복 제거, 변수/함수 네이밍 정리, 상태 위치 이동 등을 통해 코드 품질을 높이고 오류를 수정한다.

```
// 개선 전
const addItem = item ⇒ setItems([...items, item]);

// 개선 후 (불변성 유지)
setItems(prev ⇒ [...prev, item]);
```

② 문서화

README, API 문서, 화면 설계 문서를 작성하여 팀원 또는 사용자에게 프로젝트 정보를 명확히 전달하고 유지보수성을 확보한다.

8.3 금융 서비스 프로젝트 애플리케이션

이번 절에서는 금융 서비스 프로젝트 애플리케이션 예제를 실제 실행 가능한 수준으로 실습하겠습니다. 여기서는 React + Axios 기반으로 가상 은행 서비스 GTBank API를 연동하는 구조를 설계하고, 전체 소스 코드, 디렉터리 구조, 설치/실행 명령까지 실습합니다.

(1) 프로젝트 구조와 생성

① 디렉터리 구조

```
ch08/
├── node_modules/
├── public/
│   └── index.html
├── src/
│   ├── api/
│   │   └── bankApi.js
│   ├── components/
│   │   ├── Header.js
│   │   ├── AccountList.js
│   │   ├── AccountDetail.js
│   │   └── Loader.js
│   ├── pages/
│   │   ├── Home.js
│   │   └── Detail.js
```

```
|   └─ App.js
|   └─ index.js
|   └─ App.css
└─ package.json
└─ README.md
```

② 설치 및 실행 명령

```
# 1. 프로젝트 생성
npx create-react-app ch08
cd ch08

# 2. Axios 설치
npm install axios react-router-dom

# 3. 개발 서버 실행
npm start
```

③ Axios API 설정 (src/api/bankApi.js)

```
import axios from "axios";

const API_BASE = "https://api.mock.gtbank.com"; // 가상의 GTBank API

export const fetchAccounts = async () => {
  try {
    const res = await axios.get(`${API_BASE}/accounts`);
    return res.data;
  } catch (err) {
    throw err;
  }
};

export const fetchAccountDetail = async (id) => {
  try {
    const res = await axios.get(`${API_BASE}/accounts/${id}`);
```

```

    return res.data;
  } catch (err) {
    throw err;
  }
};

```

- `fetchAccounts` : 계좌 목록을 가져오는 API
- `fetchAccountDetail` : 특정 계좌 상세 정보 API

(2) 공통 컴포넌트

① Loader.js

```

function Loader() {
  return <div className="loader">로딩 중...</div>;
}

export default Loader;

```

API 요청 시 로딩 표시용 컴포넌트

② Header.js

```

import { Link } from "react-router-dom";

function Header() {
  return (
    <header>
      <h1>GTBank 금융 서비스</h1>
      <nav>
        <Link to="/">홈</Link>
      </nav>
    </header>
  );
}

```

```
export default Header;
```

페이지 상단 헤더, 홈으로 이동 링크 포함

③ AccountList.js

```
import { Link } from "react-router-dom";

function AccountList({ accounts }) {
  if (!accounts || accounts.length === 0) return <p>계좌가 없습니다.</p>;

  return (
    <ul className="account-list">
      {accounts.map((acc) => (
        <li key={acc.id}>
          <Link to={`/${detail}/${acc.id}`}>
            {acc.name} - 잔액: {acc.balance}원
          </Link>
        </li>
      ))}
    </ul>
  );
}

export default AccountList;
```

계좌 목록 표시 및 상세 화면 이동 링크 제공

④ AccountDetail.js

```
function AccountDetail({ account }) {
  if (!account) return <p>정보가 없습니다.</p>;

  return (
    <div className="account-detail">
```

```

    <h2>{account.name} 계좌 정보</h2>
    <p>계좌번호: {account.number}</p>
    <p>잔액: {account.balance}원</p>
    <p>상태: {account.status}</p>
  </div>
);
}

export default AccountDetail;

```

계좌 상세 정보 표시

(3) 페이지 컴포넌트

① Home.js

```

import React, { useEffect, useState } from "react";
import { fetchAccounts } from "../api/bankApi";
import AccountList from "../components/AccountList";
import Loader from "../components/Loader";

function Home() {
  const [accounts, setAccounts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetchAccounts()
      .then((data) => setAccounts(data))
      .catch(() => setError("계좌 정보를 가져오지 못했습니다."))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <Loader />;
  if (error) return <p>{error}</p>;

  return <AccountList accounts={accounts} />;
}

```

```
export default Home;
```

- 계좌 목록 API 호출
- 로딩/에러 처리
- AccountList 컴포넌트 전달

② Detail.js

```
import React, { useEffect, useState } from "react";
import { useParams } from "react-router-dom";
import { fetchAccountDetail } from "../api/bankApi";
import AccountDetail from "../components/AccountDetail";
import Loader from "../components/Loader";

function Detail() {
  const { id } = useParams();
  const [account, setAccount] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchAccountDetail(id)
      .then((data) => setAccount(data))
      .finally(() => setLoading(false));
  }, [id]);

  if (loading) return <Loader />;

  return <AccountDetail account={account} />;
}

export default Detail;
```

- URL 파라미터 기반 계좌 상세 API 호출
- 로딩 처리

③ App.js

```
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
import Header from "./components/Header";
import Home from "./pages/Home";
import Detail from "./pages/Detail";
import "./App.css";

function App() {
  return (
    <Router>
      <Header />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/detail/:id" element={<Detail />} />
      </Routes>
    </Router>
  );
}

export default App;
```

- SPA 구조 및 라우팅 구성
- 홈/상세 페이지 연결

④ index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./App.css";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

⑤ App.css

```
body {  
  font-family: Arial, sans-serif;  
  background: #f5f5f5;  
  margin: 0;  
  padding: 0;  
}  
  
header {  
  background: #0078d4;  
  color: white;  
  padding: 10px;  
  text-align: center;  
}  
  
header nav a {  
  color: white;  
  margin-left: 10px;  
  text-decoration: none;  
}  
  
.account-list, .account-detail {  
  max-width: 600px;  
  margin: 20px auto;  
  background: white;  
  padding: 15px;  
  border-radius: 8px;  
}  
  
.account-list li {  
  margin-bottom: 10px;  
}  
  
.loader {  
  text-align: center;  
  margin: 50px;  
}
```

